CMX-PRD-019 Notification & Communication Hub - Enterprise Edition

Version: 1.0 Enterprise Edition
Status: Authoritative Implementation Baseline
Project: CleanMateX - Laundry and Dry Cleaning SaaS
Target region: GCC first, global-ready
Languages: English and Arabic with RTL support
Prepared for: Cursor AI, Claude, Copilot, backend developers, frontend developers, QA, DevOps
Generated: 2026-05-29

Document Control

This package defines the production-grade Notification & Communication Hub for CleanMateX. It is intended to be used as the implementation source of truth. Developers must not hardcode notification behavior inside business modules. All communication must flow through this hub.

Included Package Files

File	Purpose
CMX-PRD-019_Notification_Communication_Hub_Enterprise_Edition.md	Main authoritative PRD and architecture document.
CMX-PRD-019_Notification_Communication_Hub_Enterprise_Edition.docx	Client-ready DOCX version.
019_notification_schema.sql	PostgreSQL 16 schema, indexes, catalogs, RLS hooks, and seeds.
019_notification_openapi.yaml	API contract for backend and frontend teams.
notification_event_catalog.csv	Full event catalog for product, QA, and implementation teams.
notification_template_catalog_en_ar.json	EN/AR template catalog across channels.
cursor_implementation_prompt.md	Ready prompt for Cursor/Claude/Copilot implementation.

1. Executive Summary

The Notification & Communication Hub is a central platform service responsible for transactional, operational, marketing, security, and SaaS platform communications. It supports in-app notifications, push notifications, WhatsApp Business API, SMS, email, and real-time web alerts.

This subsystem is not a simple message sender. It is an event-driven communication engine with templates, localization, preferences, consent, retry, fallback, quotas, feature flags, provider abstraction, audit logs, and observability.

Strategic Position

CleanMateX depends on notifications for customer trust, workflow speed, driver coordination, payment collection, SLA control, marketing growth, and platform monetization. A weak notification design will create operational chaos and customer complaints. A strong design becomes a competitive advantage.

Primary Goals

1. Centralize all outbound and in-app communication.
2. Remove notification logic from business modules.
3. Support AR/EN templates and RTL-safe messaging.
4. Support GCC-first WhatsApp and SMS behavior.

5. Support customer preferences and marketing consent.
6. Enforce tenant feature flags and plan quotas.
7. Provide full delivery audit and provider diagnostics.
8. Enable reliable async delivery with retries and fallback.
9. Prepare for future campaign automation and AI personalization.

Non-Goals for MVP

The following must not be built in the first implementation slice:

- AI best-send-time optimization.
- Complex omnichannel marketing journeys.
- Custom visual email builder.
- Two-way WhatsApp bot conversation engine.
- Multi-provider least-cost routing engine.

These are planned for later phases after the transactional and operational foundation is stable.

2. Architectural Principles

2.1 Event-Driven First

All business modules emit events. The notification hub consumes events and decides who receives what, through which channel, when, and in which language.

Bad pattern:

```
OrderService -> WhatsApp API
PaymentService -> SMS API
DriverService -> FCM API
```

Approved pattern:

```
Business Module -> Domain Event -> Notification Event -> Orchestrator -> Outbox -> Provider Adapter
```

2.2 Backend Enforcement

The frontend may display settings, preferences, and templates, but enforcement must happen in the backend. The backend must check:

- Tenant feature flags.
- Plan quotas.
- Recipient consent.
- Channel availability.
- Quiet hours.
- Template approval status.
- Provider health.

2.3 Multi-Tenant by Design

Every tenant-owned runtime table must include `tenant_org_id`. RLS policies must prevent tenant data leakage. Platform catalogs are global and read-only for tenants.

2.4 WhatsApp-First for GCC, Not WhatsApp-Only

WhatsApp is a primary channel in the GCC market, especially for stub customers and small laundries. However, the architecture must not depend on one provider. Every WhatsApp flow must have fallback options.

2.5 No Silent Failures

Every notification attempt must be logged. Failures must be visible in admin screens and operational dashboards.

2.6 Consent and Compliance

Transactional notifications can be sent as part of service delivery. Marketing notifications require explicit consent and opt-out support.

3. Scope

3.1 In Scope

- In-app notification center.
- Notification bell and unread count.
- FCM push notification support.
- WhatsApp Business API adapter.
- SMS adapter.
- Email adapter.
- WebSocket live alerts for admin dashboards.
- Template catalog with EN/AR versions.
- Event catalog.
- Tenant settings.
- Customer/staff preferences.
- Quiet hours.
- Retry logic.
- Fallback logic.
- Outbox pattern.
- Delivery logs.
- Usage tracking.
- Plan quota enforcement.
- Feature flag enforcement.
- Campaign foundation.
- Provider webhook handling.
- Observability and dashboards.

3.2 Out of Scope for Initial Build

- Drag-and-drop campaign journey builder.
- AI copywriting assistant.
- AI audience segmentation.
- WhatsApp conversational bot.
- Full CRM replacement.
- External marketing platform replacement.

4. Stakeholders and Recipients

Recipient Type	Description	Primary Channels	Notes
Customer - Guest	No app, may not have phone.	Printed receipt, QR, SMS if phone exists	Limited digital capability.
Customer - Stub	POS-created minimal profile with phone.	WhatsApp, SMS, tracking link	Most important early GCC flow.
Customer - Full	App user with verified OTP.	In-app, push, WhatsApp	Best digital experience.
B2B Contact	Corporate/hotel/restaurant contact.	Email, WhatsApp, PDF link	Formal documents preferred.
Driver	Delivery staff.	Push, in-app, critical SMS	Needs urgent operational alerts.
Staff	Reception, preparation, processing, QA, packing.	In-app, push, web live alert	Task-focused.
Branch Manager	Branch operational manager.	In-app, push, email	SLA, staff, issues.
Tenant Admin	Laundry owner/admin.	In-app, email, WhatsApp optional	Billing, limits, system alerts.
Platform Admin	CleanMateX HQ.	In-app, email, dashboard	Provider, queue, tenant health.

5. Notification Categories

The system must use stable category codes. These codes are used for filtering, templates, permissions, analytics, and recipient preferences.

Category Code	Name	Description
ASSEMBLY	Assembly	Global category for notification grouping and filtering.
CUSTOMER	Customer	Global category for notification grouping and filtering.
DELIVERY	Delivery	Global category for notification grouping and filtering.
GIFTCARD	Giftcard	Global category for notification grouping and filtering.
INVENTORY	Inventory	Global category for notification grouping and filtering.
INVOICE	Invoice	Global category for notification grouping and filtering.
LOYALTY	Loyalty	Global category for notification grouping and filtering.
MACHINE	Machine	Global category for notification grouping and filtering.
MARKETING	Marketing	Global category for notification grouping and filtering.
MEMBERSHIP	Membership	Global category for notification grouping and filtering.
ORDER	Order	Global category for notification grouping and filtering.
ORDER_ITEM	Order Item	Global category for notification grouping and filtering.
PACKING	Packing	Global category for notification grouping and filtering.
PAYMENT	Payment	Global category for notification grouping and filtering.
PICKUP	Pickup	Global category for notification grouping

Category Code	Name	Description
		and filtering.
PLAN_LIMIT	Plan Limit	Global category for notification grouping and filtering.
PREPARATION	Preparation	Global category for notification grouping and filtering.
PROCESSING	Processing	Global category for notification grouping and filtering.
QA	Qa	Global category for notification grouping and filtering.
REFUND	Refund	Global category for notification grouping and filtering.
SECURITY	Security	Global category for notification grouping and filtering.
STAFF	Staff	Global category for notification grouping and filtering.
STATEMENT	Statement	Global category for notification grouping and filtering.
SUBSCRIPTION	Subscription	Global category for notification grouping and filtering.
SYSTEM	System	Global category for notification grouping and filtering.
WALLET	Wallet	Global category for notification grouping and filtering.
WORKFLOW	Workflow	Global category for notification grouping and filtering.

6. Channel Strategy

Channel	Purpose	Strength	Weakness	Use For
IN_APP	Permanent notification center.	Reliable, auditable, no external cost.	Requires user login.	All user-visible events.
PUSH	Immediate mobile alert.	Fast, cheap.	Token may expire, app may be disabled.	Customer, driver, staff alerts.
WHATSAPP	GCC-friendly customer communication.	High open rate, familiar UX.	Paid, template approval, provider limits.	Stub customers, order status, invoices.
SMS	Critical fallback.	Works without smartphone/data.	Costly, limited content.	OTP, fallback, urgent delivery.
EMAIL	Formal documents and admin communication.	Good for invoices/statements.	Slow engagement for B2C.	B2B, tenant admin, platform alerts.
WEB_SOCKET	Live dashboard alert.	Real-time operations.	Online session required.	Web Admin, HQ dashboards.

Channel Selection Rules

1. Always create IN_APP record for authenticated users unless the event is internal-only.
2. Use PUSH for app users when token exists and preference allows.
3. Use WHATSAPP for stub customers and customer-visible transactional events when tenant has entitlement.
4. Use SMS only for OTP, fallback, or critical events due to cost.
5. Use EMAIL for documents, B2B, tenant admin, platform notices, and formal audit communication.
6. Use WEB_SOCKET only as a live companion, never as the only durable notification.

7. Event Catalog Summary

The complete machine-readable event catalog is included in notification_event_catalog.csv. Each event has category, name, description, default recipients, channels, priority, consent behavior, and idempotency key pattern.

Sample Events

Category	Event Code	Name	Channels	Priority
ORDER	order.created	Order created	IN_APP,PUSH,WHATSAPP	NORMAL
ORDER	order.quick_drop.created	Quick drop order created	IN_APP,WHATSAPP	NORMAL
ORDER	order.preparation.required	Preparation required	IN_APP,PUSH	HIGH
ORDER	order.prepared	Order prepared	IN_APP,PUSH,WHATSAPP	NORMAL
ORDER	order.processing.started	Processing started	IN_APP,PUSH	NORMAL
ORDER	order.ready	Order ready	IN_APP,PUSH,WHATSAPP,SMS	HIGH
ORDER	order.delayed	Order delayed	IN_APP,PUSH,WHATSAPP	HIGH
ORDER	order.cancelled	Order cancelled	IN_APP,PUSH,WHATSAPP,EMAIL	HIGH
ORDER	order.closed	Order closed	IN_APP,EMAIL	NORMAL
ORDER	order.priority.changed	Order priority changed	IN_APP,PUSH	NORMAL
ORDER	order.split.created	Split order created	IN_APP,PUSH	HIGH
ORDER	order.issue.created	Order issue created	IN_APP,PUSH,WHATSAPP	HIGH
ORDER	order.issue.resolved	Order issue resolved	IN_APP,PUSH,WHATSAPP	NORMAL
ORDER_ITEM	order.item.added	Order item added	IN_APP	LOW
ORDER_ITEM	order.item.removed	Order item removed	IN_APP	NORMAL
ORDER_ITEM	order.item.missing	Order item missing	IN_APP,PUSH,WHATSAPP	URGENT
ORDER_ITEM	order.item.damaged	Order item damaged	IN_APP,PUSH,WHATSAPP	URGENT
ORDER_ITEM	order.item.rework.required	Item rework required	IN_APP,PUSH	HIGH
PREPARATION	preparation.task.created	Preparation task created	IN_APP,PUSH	HIGH
PREPARATION	preparation.task.completed	Preparation task completed	IN_APP,PUSH	NORMAL
PROCESSING	processing.step.completed	Processing step completed	IN_APP	LOW
ASSEMBLY	assembly.started	Assembly started	IN_APP,PUSH	NORMAL
ASSEMBLY	assembly.exception	Assembly exception	IN_APP,PUSH	URGENT
ASSEMBLY	assembly.completed	Assembly completed	IN_APP,PUSH	NORMAL
QA	qa.started	QA started	IN_APP	NORMAL
QA	qa.passed	QA passed	IN_APP,PUSH	NORMAL
QA	qa.failed	QA failed	IN_APP,PUSH,WHATSAPP	HIGH
PACKING	packing.completed	Packing completed	IN_APP,PUSH	NORMAL
PICKUP	pickup.scheduled	Pickup scheduled	IN_APP,PUSH,WHATSA	NORMAL

Category	Event Code	Name	Channels	Priority
			PP	
PICKUP	pickup.reminder	Pickup reminder	PUSH,WHATSAPP,SMS	NORMAL
PICKUP	pickup.driver_assigned	Pickup driver assigned	PUSH,WHATSAPP	NORMAL
PICKUP	pickup.completed	Pickup completed	PUSH,WHATSAPP,IN_APP	NORMAL
DELIVERY	delivery.assigned	Delivery assigned	PUSH,IN_APP,WHATSAPP	HIGH
DELIVERY	delivery.out_for_delivery	Out for delivery	PUSH,WHATSAPP,SMS	HIGH
DELIVERY	delivery.otp_generated	Delivery OTP generated	PUSH,WHATSAPP,SMS	URGENT
DELIVERY	delivery.arrived	Driver arrived	PUSH,WHATSAPP,SMS	URGENT
DELIVERY	delivery.failed	Delivery failed	PUSH,WHATSAPP,IN_APP	HIGH
DELIVERY	delivery.delivered	Delivered	PUSH,WHATSAPP,EMAIL,IN_APP	NORMAL
DELIVERY	delivery.collected	Collected	PUSH,WHATSAPP,IN_APP	NORMAL
PAYMENT	payment.requested	Payment requested	PUSH,WHATSAPP,SMS,EMAIL	HIGH

8. Event Payload Contract

Every emitted notification event must follow this shape:

```
{
 "tenant_org_id": "uuid",
 "event_code": "order.ready",
 "source_module": "orders",
 "source_entity_type": "ORDER",
 "source_entity_id": "uuid",
 "idempotency_key": "tenant:event:entity:recipient-scope",
 "payload_jsonb": {
  "orderId": "uuid",
  "orderNo": "ORD-2026-0001",
  "customerId": "uuid",
  "customerName": "Ali",
  "phone": "+968...",
  "lang": "ar",
  "totalAmount": "5.250",
  "currency": "OMR",
  "trackingUrl": "https://..."
 }
}
```

Required Payload Fields by Event Family

Event Family	Required Fields
ORDER	orderId, orderNo, customerId/customer reference, branchId, status, trackingUrl
DELIVERY	orderId, orderNo, driverId, driverName, destination, otpCode when applicable
PAYMENT	orderId, orderNo, amount, currency, paymentStatus, paymentMethod
INVOICE	invoiceId, invoiceNo, orderId, customerId, invoiceUrl
B2B STATEMENT	statementId, statementNo, companyName, statementUrl
CAMPAIGN	campaignId, campaignName, recipientId, promoCode if

Event Family	Required Fields
	applicable
SECURITY	userId, actionName, ipAddress, deviceInfo, riskScore
SYSTEM	incidentId or maintenanceId, startAt, endAt, impactLevel

9. Runtime Flow

9.1 Transactional Flow - Order Ready

1. Staff completes packing and marks order READY.
2. Order module commits order status and writes domain history.
3. Order module emits event: order.ready.
4. Notification event is inserted with idempotency key.
5. BullMQ notification-events worker picks event.
6. Orchestrator resolves recipients.
7. Policy resolver checks tenant feature flags and plan quotas.
8. Preference resolver checks customer preferences and consent.
9. Template renderer resolves EN/AR template.
10. Channel router creates outbox rows.
11. Dispatch worker sends provider messages asynchronously.
12. Delivery log records every attempt.
13. Provider webhooks update delivery/read status.
14. Notification center shows durable in-app record.

9.2 Marketing Flow - Campaign

1. Marketing user creates campaign draft.
2. Segment preview calculates eligible recipients.
3. System excludes recipients without marketing consent.
4. Campaign is approved by authorized role.
5. Campaign is scheduled or launched.
6. Campaign worker chunks recipients.
7. Quota service validates limits.
8. Outbox rows are created.
9. Dispatch worker sends messages with safe-send caps.
10. Opens, clicks, bounces, and conversions are tracked.

9.3 Provider Webhook Flow

1. Provider sends webhook callback.
2. Webhook signature is verified.
3. Provider message ID is resolved to outbox row.
4. Delivery status is updated.
5. Delivery log entry is inserted.
6. Usage counters are updated.
7. Failed messages may trigger fallback.

10. State Machines

10.1 Outbox State Machine

QUEUED -> PROCESSING -> SENT -> DELIVERED -> READ
QUEUED -> PROCESSING -> FAILED_TEMPORARY -> RETRYING -> PROCESSING
QUEUED -> PROCESSING -> FAILED_PERMANENT
QUEUED -> SKIPPED
QUEUED -> CANCELLED

10.2 Campaign State Machine

DRAFT -> PENDING_APPROVAL -> APPROVED -> SCHEDULED -> RUNNING -> COMPLETED
DRAFT -> CANCELLED
PENDING_APPROVAL -> DRAFT
APPROVED -> CANCELLED
RUNNING -> PAUSED -> RUNNING
RUNNING -> FAILED

10.3 Template Version State Machine

DRAFT -> APPROVED -> RETIRED
DRAFT -> RETIRED

Rules:

- Only one active approved template version per template.
- WhatsApp templates may require provider approval before activation.
- Retired templates remain available for historical delivery logs.

11. Database Design

The complete SQL is included in 019_notification_schema.sql. This document summarizes the model.

11.1 Global Catalogs

Table	Purpose
sys_notification_categories_cd	Stable categories.
sys_notification_channels_cd	Channel metadata and capabilities.
sys_notification_providers_cd	Provider catalog.
sys_notification_events_cd	Global event catalog.
sys_notification_templates	Template master.
sys_notification_template_versions	Versioned template lifecycle.
sys_notification_template_channels	Channel/language-specific body.

11.2 Tenant Runtime Tables

Table	Purpose
org_notification_settings_cf	Tenant notification settings.
org_notification_preferences	User/customer/staff preferences.
org_notification_events	Runtime notification events.
org_notifications	Durable in-app notification center records.
org_notification_outbox	Async dispatch outbox.
org_notification_delivery_log	Provider attempts and results.
org_notification_campaigns	Campaign master.
org_notification_campaign_recipients	Campaign target tracking.
org_notification_usage_daily	Usage and cost aggregation.
org_notification_audit	Admin changes and sensitive actions.

11.3 Indexing Strategy

- Outbox dispatch index: (status, scheduled_at, priority).
- Recipient notification index: (tenant_org_id, recipient_type, recipient_id, status, created_at desc).
- Event entity index: (tenant_org_id, source_entity_type, source_entity_id).
- Provider message lookup: (provider_code, provider_message_id).
- Usage daily unique index for channel/provider/date aggregation.

11.4 Partitioning Recommendation

For large tenants, partition high-volume tables by month:

- org_notification_outbox
- org_notification_delivery_log
- org_notification_events

Initial MVP may run without partitions, but the schema must be partition-ready.

12. Multi-Tenant RLS Requirements

Every org_* table must be protected by tenant context.

Example policy pattern:

```
create policy tenant_isolation_org_notification_events
on public.org_notification_events
using (tenant_org_id = current_setting('app.current_tenant_id', true)::uuid);
```

RLS Rules

1. Tenant users can only access rows matching their tenant_org_id.
2. Platform admin access must use a controlled service role, never normal tenant JWT.
3. Provider webhooks must not bypass validation. They may use service role but must resolve tenant and provider message identity.
4. Campaign operations must verify tenant admin/marketing permissions.
5. Delivery logs can be visible to tenant admins but sensitive provider payloads may be masked.

13. Feature Flags and Plan Limits

13.1 Feature Flags

Flag	Description	Default MVP
notifications_enabled	Enables notification hub.	true
in_app_notifications_enabled	Enables notification center.	true
push_notifications_enabled	Enables FCM push.	true
whatsapp_notifications_enabled	Enables WhatsApp sending.	false until provider configured
sms_notifications_enabled	Enables SMS sending.	false until provider configured
email_notifications_enabled	Enables email.	true
notification_campaigns_enabled	Enables campaigns.	false
notification_templates_editable	Allows tenant template override.	false
notification_quiet_hours_enabled	Enables quiet hours.	true
notification_webhooks_enabled	Enables provider callback handling.	true

13.2 Plan Quotas

Plan	Push/month	WhatsApp/month	SMS/month	Email/month	Campaigns
Free Trial	200	50	20	100	No
Starter	1000	500	500	1000	No
Growth	5000	3000	2000	5000	Basic
Pro	25000	15000	10000	50000	Advanced
Enterprise	Custom	Custom	Custom	Custom	Advanced

13.3 Quota Enforcement

Quota is checked before creating outbox rows for paid channels. Critical transactional messages may be allowed to exceed quota if platform policy enables emergency overage billing.

Recommended behavior:

- Transactional over quota: send if critical, log billable overage.
- Marketing over quota: block.
- Provider unavailable: fallback if allowed.

- Tenant channel disabled: skip and log policy skip.

14. Preferences, Consent, and Quiet Hours

14.1 Preferences

Recipients may configure:

- Language.
- Push on/off.
- WhatsApp on/off.
- SMS on/off.
- Email on/off.
- Marketing consent.
- Quiet hours.
- Category-level preferences.

14.2 Transactional vs Marketing

Type	Consent Required	Example
Transactional	No, but channel preference applies where practical	Order ready, invoice issued
Operational	No for staff/users within tenant employment context	Task assigned, SLA warning
Security	No	Suspicious login
Marketing	Yes	Promotion, win-back campaign

14.3 Quiet Hours

Quiet hours delay non-urgent messages. Critical messages bypass quiet hours.

Bypass examples:

- Delivery OTP.
- Driver arrived.
- Security suspicious activity.
- Payment failure for active checkout.
- System outage for tenant admin.

15. Template System

The template system must support channel-specific formatting. A WhatsApp template is not the same as an email template.

15.1 Template Variables

Variables use double curly braces:

```
{{customerName}}
{{orderNo}}
{{totalAmount}}
{{currency}}
{{trackingUrl}}
```

15.2 Template Validation

Before activation:

1. All required variables must exist in event payload schema.
2. SMS body must not exceed configured length unless multi-part SMS is allowed.
3. WhatsApp templates must have approved provider template code if required.
4. Arabic text must be RTL-safe.
5. Template must render with sample payload.

15.3 Template Fallback

If Arabic template is missing:

1. Use tenant default language.
2. Use English fallback.
3. Log missing translation warning.

Production should not allow missing Arabic for customer-facing GCC templates.

16. Provider Adapter Contract

Every provider adapter must implement the same contract.

```ts
export interface NotificationProviderAdapter {
  providerCode: string;
  channelCode: NotificationChannelCode;

  validateConfig(config: ProviderConfig): Promise<ValidationResult>;

  send(request: ProviderSendRequest): Promise<ProviderSendResult>;

  parseWebhook(rawBody: unknown, headers: Record<string, string>): Promise<ProviderWebhookEvent>;

  verifyWebhookSignature(rawBody: Buffer, headers: Record<string, string>, secret: string): boolean;
}
```

Send Request

```ts
export interface ProviderSendRequest {
  tenantOrgId: string;
  outboxId: string;
  destination: string;
  title?: string;
  body: string;
  templateCode?: string;
  providerTemplateCode?: string;
  variables?: Record<string, unknown>;
  attachments?: Array<{ fileName: string; url: string; contentType: string }>;
  metadata?: Record<string, unknown>;
}
```

Send Result

```ts
export interface ProviderSendResult {
  success: boolean;
  providerMessageId?: string;
  providerStatus?: string;
  rawResponse?: unknown;
  errorCode?: string;
  errorMessage?: string;
  retryable?: boolean;
```

```
  costAmount?: string;
  costCurrency?: string;
  latencyMs?: number;
}
```

17. NestJS Module Design

17.1 Folder Structure

```
backend/src/modules/notifications/
  notifications.module.ts
  notifications.constants.ts

  api/
    notifications.controller.ts
    notification-settings.controller.ts
    notification-preferences.controller.ts
    notification-templates.controller.ts
    notification-events.controller.ts
    notification-campaigns.controller.ts
    notification-delivery-log.controller.ts
    provider-webhooks.controller.ts

  application/
    notification-orchestrator.service.ts
    recipient-resolver.service.ts
    template-renderer.service.ts
    channel-router.service.ts
    policy-resolver.service.ts
    preference-resolver.service.ts
    quota.service.ts
    delivery-log.service.ts
    notification-center.service.ts
    campaign.service.ts
    provider-webhook.service.ts

  channels/
    provider-adapter.interface.ts
    fcm-push.adapter.ts
    meta-whatsapp.adapter.ts
    twilio-sms.adapter.ts
    local-sms.adapter.ts
    sendgrid-email.adapter.ts
    aws-ses-email.adapter.ts
    websocket.adapter.ts
    in-app.adapter.ts

  workers/
    notification-event.worker.ts
    notification-dispatch.worker.ts
    notification-retry.worker.ts
    notification-campaign.worker.ts
    provider-webhook.worker.ts
    notification-digest.worker.ts

  repositories/
    notification-events.repository.ts
    notification-outbox.repository.ts
    notification-template.repository.ts
    notification-settings.repository.ts
    notification-preferences.repository.ts
    notification-campaign.repository.ts
    notification-usage.repository.ts
```

dto/
types/
validators/
tests/

17.2 Service Responsibilities

Service	Responsibility
NotificationOrchestratorService	Main flow from event to outbox.
RecipientResolverService	Determines recipients from event and payload.
TemplateRendererService	Loads template, validates variables, renders text.
ChannelRouterService	Determines channels and fallback.
PolicyResolverService	Applies feature flags, plan entitlements, quiet hours.
PreferenceResolverService	Applies recipient preferences and consent.
QuotaService	Checks and records usage.
DeliveryLogService	Records provider attempts and statuses.
NotificationCenterService	Manages in-app notifications.
CampaignService	Creates, approves, launches, and tracks campaigns.

18. BullMQ Queue Design

Queue	Purpose	Producer	Worker	Concurrency MVP	Concurrency Scale
notification-events	Process raw events.	Business modules	notification-event.worker	5	50
notification-dispatch	Send outbox messages.	Orchestrator	notification-dispatch.worker	10	200
notification-retry	Retry temporary failures.	Dispatch worker	notification-retry.worker	5	100
notification-campaigns	Chunk and queue campaigns.	Campaign service	notification-campaign.worker	2	20
provider-webhooks	Process provider callbacks.	Webhook controller	provider-webhook.worker	5	100
notification-digest	Create digest summaries.	Scheduler	notification-digest.worker	2	20

Retry Backoff

Attempt 1: immediate
Attempt 2: after 1 minute
Attempt 3: after 5 minutes
Attempt 4: after 15 minutes
Attempt 5: after 1 hour
After max attempts: FAILED_PERMANENT

Dead Letter Handling

Poison messages must be moved to failed state with error payload. They must not block the queue.

19. API Specification Summary

The full OpenAPI contract is included in 019_notification_openapi.yaml.

API Groups

Group	Endpoints
Notifications	list, unread count, mark read, archive, delete
Settings	get/update tenant notification settings

Group	Endpoints
Preferences	get/update current recipient preferences
Templates	global and tenant template management
Events	emit event, inspect event processing
Delivery Logs	search and diagnose provider deliveries
Campaigns	create, approve, launch, pause, inspect
Provider Webhooks	receive delivery/read/failure callbacks

Standard Errors

Code	Meaning
NOTIFICATION_FEATURE_DISABLED	Tenant feature flag disabled.
CHANNEL_DISABLED	Channel not enabled for tenant.
CHANNEL_NOT_ALLOWED_BY_PLAN	Plan entitlement blocks channel.
QUOTA_EXCEEDED	Monthly quota exceeded.
TEMPLATE_NOT_FOUND	No template for event/channel/language.
TEMPLATE_NOT_APPROVED	Template exists but not approved.
CONSENT_REQUIRED	Marketing consent missing.
PROVIDER_CONFIG_MISSING	Provider not configured.
PROVIDER_TEMPORARY_FAILURE	Retryable provider failure.
PROVIDER_PERMANENT_FAILURE	Non-retryable provider failure.

20. UI Specifications - Web Admin

20.1 Notification Bell

Location: top navigation header.

Features:

- Unread count badge.
- Dropdown recent notifications.
- Mark all as read.
- Link to notification center.
- Filter by critical/unread.

20.2 Notification Center

Route: /notifications

Sections:

- All.
- Unread.
- Orders.
- Delivery.
- Payments.
- System.
- Archived.

Columns/list fields:

- Priority indicator.

- Category.
- Title.
- Body preview.
- Related entity link.
- Created time.
- Read/archive actions.

20.3 Notification Settings

Route: /settings/notifications

Tabs:

1. Channels.
2. Fallback rules.
3. Quiet hours.
4. Customer preferences defaults.
5. Staff alerts.
6. Usage and quotas.
7. Provider health.

20.4 Template Management

Route: /settings/notification-templates

For MVP, tenant editing may be hidden. Platform HQ manages templates.

Fields:

- Template code.
- Category.
- Event.
- Channel.
- Language.
- Title.
- Body.
- Variables.
- Preview sample.
- Status.
- Version.

20.5 Delivery Logs

Route: /settings/notification-delivery-log

Filters:

- Date range.
- Channel.
- Provider.
- Status.
- Recipient.
- Order/invoice/payment reference.

- Error code.

Actions:

- View payload.
- Retry failed.
- Send fallback.
- Export CSV.

20.6 Campaigns

Route: /marketing/campaigns

MVP disabled unless notification_campaigns_enabled is true.

Pages:

- Campaign list.
- Campaign create/edit.
- Segment preview.
- Approval.
- Launch progress.
- Results dashboard.

21. UI Specifications - Customer App

21.1 Notification Bell

- Appears in home screen and order tracking screen.
- Shows unread count.
- Opens notification center.

21.2 Notification Center

Tabs:

- All.
- Orders.
- Payments.
- Offers.
- System.

Actions:

- Mark as read.
- Delete/archive.
- Open related order/invoice.

21.3 Preferences Screen

Route: profile -> notification preferences.

Fields:

- Preferred language.
- Push enabled.

- WhatsApp enabled.
- SMS enabled.
- Email enabled.
- Marketing consent.
- Quiet hours.

Rules:

- Critical security and delivery OTP cannot be fully disabled.
- Marketing opt-out must be simple and immediate.

22. UI Specifications - Driver App

Driver notifications must be operational and low-friction.

Screens:

- Job alert card.
- Pickup reminder.
- Out-for-delivery task update.
- Delivery OTP prompt.
- Failed delivery escalation.

Rules:

- Critical job alerts use push.
- If push fails repeatedly, SMS can be used for urgent driver tasks.
- Driver app must sync notifications after reconnecting from offline mode.

23. UI Specifications - Platform HQ

Platform HQ controls global communication infrastructure.

Pages:

1. Global Template Library.
2. Provider Configurations.
3. Provider Health Dashboard.
4. Tenant Usage and Quotas.
5. Failed Notification Monitor.
6. WhatsApp Template Approval Tracker.
7. Global Event Catalog.
8. Platform Campaigns and Announcements.

HQ must see cross-tenant metrics but must mask personal content unless support access is explicitly justified and audited.

24. Permissions Matrix

Capability	Platform Admin	Tenant Admin	Branch Manager	Staff	Driver	Customer
View own notifications	Yes	Yes	Yes	Yes	Yes	Yes
Manage own preferences	Yes	Yes	Yes	Yes	Yes	Yes

Capability	Platform Admin	Tenant Admin	Branch Manager	Staff	Driver	Customer
View tenant delivery logs	Yes	Yes	Yes limited	No	No	No
Retry failed notification	Yes	Yes	Limited	No	No	No
Configure tenant channels	Yes	Yes	No	No	No	No
Manage global templates	Yes	No	No	No	No	No
Manage tenant templates	Yes	Plan-based	No	No	No	No
Create campaign	Yes	Yes if enabled	Marketing role only	No	No	No
Approve campaign	Yes	Yes with permission	No	No	No	No
View provider config secrets	Restricted	No	No	No	No	No
View usage/quota	Yes	Yes	Yes limited	No	No	No

25. Security and Privacy

25.1 Security Controls

- JWT authentication.
- RBAC authorization.
- RLS isolation.
- Signed provider webhooks.
- Idempotency keys.
- Secrets stored outside database when possible.
- Sensitive provider payload masking.
- Audit logs for admin actions.
- Minimal PII in logs.
- Rate limiting on event creation and campaign endpoints.

25.2 Privacy Controls

- Marketing consent required.
- Opt-out immediately honored.
- Customer data export includes notification preferences and in-app notifications.
- Right-to-delete must anonymize or delete personal notification content according to retention policy.
- Delivery logs may be retained for audit but personal body content should have retention limits.

25.3 Provider Webhook Security

Every provider webhook must:

1. Verify signature.
2. Validate timestamp if provided.
3. Resolve provider message ID.
4. Reject unknown provider/config.
5. Be idempotent.
6. Log request ID and result.

26. Observability

26.1 Metrics

Metric	Description	Target
notification_event_processing_latency_ms	Time from event insert to outbox creation.	p95 < 500ms
notification_dispatch_latency_ms	Time from scheduled_at to provider attempt.	p95 < 10s push, <30s WhatsApp
notification_delivery_success_rate	Delivered/sent rate by channel.	>98% transactional
notification_provider_failure_rate	Provider failures by provider.	Alert >5% over 5 min
notification_queue_depth	Pending queue jobs.	Alert by threshold
notification_retry_count	Retries by channel/provider.	Watch trend
notification_cost_amount	Cost by tenant/channel/provider.	Used for billing
campaign_opt_out_rate	Unsubscribe rate.	Alert abnormal spikes

26.2 Dashboards

- Tenant delivery dashboard.
- Provider health dashboard.
- Queue health dashboard.
- Campaign performance dashboard.
- Cost and quota dashboard.
- Critical failures dashboard.

26.3 Alerts

- Provider failure rate above threshold.
- Queue backlog high.
- Retry storm detected.
- Webhook signature failures spike.
- Campaign bounce rate high.
- SMS/WhatsApp quota exceeded.
- Outbox stuck in PROCESSING longer than threshold.

27. Performance and SLO

Operation	Target
Create notification event	p95 < 150ms
Event to outbox rows	p95 < 500ms
In-app notification visible	p95 < 2 seconds
Push dispatch	p95 < 10 seconds
WhatsApp dispatch	p95 < 30 seconds
SMS dispatch	p95 < 20 seconds
Email dispatch	p95 < 60 seconds
Campaign throughput	100k notifications/hour scalable target
Core notification API availability	99.9%

Scaling Rules

- API remains stateless.
- Workers scale horizontally by queue depth.
- Provider rate limits must be respected per tenant and per provider.
- Campaigns must be chunked.

- Large broadcast campaigns must not block transactional notifications.

28. Testing Strategy

28.1 Unit Tests

Required coverage:

- Template rendering.
- Variable validation.
- Channel routing.
- Preference resolver.
- Policy resolver.
- Quota service.
- Retry calculation.
- Provider adapter response parsing.

28.2 Integration Tests

Required scenarios:

1. Event creates in-app notification.
2. Event creates push outbox row.
3. WhatsApp fallback to SMS on retryable failure.
4. Marketing event blocked without consent.
5. Quota blocks marketing SMS.
6. Critical OTP bypasses quiet hours.
7. Template missing Arabic falls back correctly and logs warning.
8. Provider webhook updates outbox status.
9. Duplicate idempotency key does not duplicate send.
10. Tenant A cannot read Tenant B logs.

28.3 E2E Tests

Required journeys:

- Quick drop -> preparation required -> customer WhatsApp tracking link.
- Order ready -> push/in-app/WhatsApp delivered.
- Out for delivery -> OTP SMS fallback.
- Invoice issued -> email and WhatsApp PDF link.
- Payment failed -> customer retry link.
- Campaign launch -> consent filtering -> delivery stats.

28.4 Load Tests

Use k6 for:

- 100 orders/min creating order.ready events.
- 10k push notifications in 10 minutes.
- 100k campaign recipients in one hour.
- Provider failure simulation with retries.
- Webhook burst handling.

29. Acceptance Criteria

The feature is not accepted unless all criteria are met.

Functional

- All notifications originate from events, not direct provider calls.
- In-app notifications are durable and queryable.
- Push, WhatsApp, SMS, and email have adapter abstraction.
- EN/AR template rendering works.
- Tenant settings control channels.
- Recipient preferences are applied.
- Marketing consent is enforced.
- Quiet hours are applied.
- Retry and fallback are implemented.
- Delivery logs exist for every provider attempt.
- Provider webhooks update statuses.
- Usage counters are recorded.
- Quotas are enforced.

Non-Functional

- RLS prevents cross-tenant access.
- API uses RBAC permissions.
- Provider secrets are not exposed in UI/logs.
- Queue failures do not break order creation.
- Transactional messages are not blocked by marketing campaigns.
- Observability dashboards exist.
- Core tests pass in CI.

30. Implementation Roadmap

Phase 1 - Transactional Foundation

Deliver:

- SQL schema.
- Event catalog seeds.
- Template catalog seeds.
- Notification event API.
- In-app notification center.
- FCM push adapter.
- Outbox and dispatch worker.
- Delivery logs.
- Tenant settings.
- Basic admin logs screen.

Events:

- order.created
- order.ready
- delivery.out_for_delivery

- delivery.otp_generated
- delivery.delivered
- payment.received
- payment.failed
- invoice.issued

Phase 2 - Operational Alerts

Deliver:

- Staff task alerts.
- QA/assembly exception alerts.
- Inventory alerts.
- Machine maintenance alerts.
- WebSocket live dashboard alerts.

Phase 3 - Preferences and Governance

Deliver:

- Customer preference UI.
- Staff preference UI.
- Marketing consent.
- Quiet hours.
- Quota dashboards.

Phase 4 - Campaign Engine

Deliver:

- Campaign CRUD.
- Segment preview.
- Consent filtering.
- Approval flow.
- Campaign launch workers.
- Campaign analytics.

Phase 5 - Provider Optimization

Deliver:

- Multi-provider routing.
- Provider failover.
- Cost tracking.
- Provider health scoring.

Phase 6 - AI Enhancements

Deliver later only:

- Best send time.
- Churn-risk targeting.
- Message personalization.
- Campaign ROI auto-stop.

31. Cursor / Claude / Copilot Implementation Rules

AI coding assistants must follow these rules:

1. Do not send WhatsApp/SMS/email directly from OrderService, PaymentService, DeliveryService, or InvoiceService.
2. Always emit a notification event or call NotificationOrchestrator.
3. Every send attempt must create or update outbox and delivery log records.
4. Use idempotency keys for all events.
5. Enforce tenant feature flags before creating paid-channel outbox rows.
6. Enforce marketing consent before campaign messages.
7. Keep provider adapters interchangeable.
8. Never expose provider secrets to frontend.
9. Use DTO validation for all APIs.
10. Use RBAC guards for admin actions.
11. Use RLS-safe queries and tenant_org_id in all tenant data.
12. Write tests for retry, fallback, consent, and tenant isolation.

32. Risks and Mitigations

Risk	Impact	Mitigation
WhatsApp template approval delay	Customer notifications delayed	Start with SMS/push fallback and approve templates early.
Provider outage	Failed notifications	Adapter abstraction, fallback, provider health monitor.
Campaign spam	Customer churn, legal issues	Consent, safe-send caps, approval workflow.
Quota abuse	Cost leakage	Usage counters, quotas, alerts, billing overage.
Duplicate messages	Customer annoyance	Idempotency keys and unique constraints.
Cross-tenant data leak	Critical security incident	RLS tests, tenant-scoped repositories.
Queue backlog	Delayed operations	Worker autoscaling and priority queues.
Arabic formatting issues	Poor GCC UX	RTL testing and AR template QA.

33. Developer Checklist

Backend

- ☐ Apply schema migration.
- ☐ Seed channels, categories, events, templates.
- ☐ Build NotificationModule.
- ☐ Build event API.
- ☐ Build orchestrator.
- ☐ Build template renderer.
- ☐ Build channel router.
- ☐ Build policy/preference resolvers.
- ☐ Build quota service.
- ☐ Build provider adapters.
- ☐ Build BullMQ workers.
- ☐ Build webhook controller.
- ☐ Add RBAC guards.

- ☐ Add integration tests.

Web Admin

- ☐ Add notification bell.
- ☐ Add notification center.
- ☐ Add settings pages.
- ☐ Add delivery logs page.
- ☐ Add template preview page.
- ☐ Add usage/quota dashboard.

Customer App

- ☐ Add notification bell.
- ☐ Add notification center.
- ☐ Add notification preferences.
- ☐ Add FCM token registration.
- ☐ Add deep links from notifications.

Driver App

- ☐ Add push token registration.
- ☐ Add job alert notifications.
- ☐ Add offline sync for notification center.

QA

- ☐ Test EN/AR rendering.
- ☐ Test fallback.
- ☐ Test retry.
- ☐ Test consent.
- ☐ Test quotas.
- ☐ Test RLS.
- ☐ Test provider webhook.

34. Appendix A - Priority Matrix

Priority	Delivery Timing	Quiet Hours	Retry	Example
LOW	May be delayed/digested	Respect	Low retry	Marketing tip
NORMAL	Send normally	Respect	Standard	Order created
HIGH	Send quickly	Usually bypass only if operational	Standard	Order ready
URGENT	Send immediately	Bypass	Aggressive	Driver arrived, OTP
CRITICAL	Send immediately, multiple channels	Bypass	Aggressive + alert admin	Security issue, provider outage

35. Appendix B - Digest Rules

Digest notifications reduce spam.

Examples:

- Inventory low stock digest.
- Machine maintenance digest.
- Daily branch operations summary.

- Campaign performance daily summary.

Digest rules:

1. Never digest OTP/security critical messages.
2. Digest operational low/normal alerts when configured.
3. Include count, top 5 items, and link to full list.
4. Respect tenant language and admin preferences.

36. Appendix C - Recommended MVP Templates

Minimum templates required before production pilot:

Template	Channels
ORDER_CREATED	In-app, Push, WhatsApp, SMS
ORDER_READY	In-app, Push, WhatsApp, SMS
ORDER_DELAYED	In-app, Push, WhatsApp
DELIVERY_OUT_FOR_DELIVERY	Push, WhatsApp, SMS
DELIVERY_OTP_GENERATED	Push, WhatsApp, SMS
DELIVERY_DELIVERED	In-app, Push, WhatsApp
PAYMENT_REQUESTED	Push, WhatsApp, SMS, Email
PAYMENT_RECEIVED	In-app, Push, WhatsApp, Email
PAYMENT_FAILED	Push, WhatsApp, SMS
INVOICE_ISSUED	In-app, WhatsApp, Email
CUSTOMER_STUB_CREATED	WhatsApp, SMS
PLAN_LIMIT_REACHED	In-app, Email
SECURITY_SUSPICIOUS_ACTIVITY	Push, Email, In-app

37. Final Design Position

The Notification & Communication Hub must be built as a platform subsystem. It is not optional glue code. It is part of CleanMateX's SaaS infrastructure and directly affects customer experience, operational efficiency, billing, compliance, and monetization.

The approved implementation path is:

Event-first -> Outbox-first -> Template-first -> Provider-abstraction -> Audit-first -> Quota-aware -> AI-ready